WASP SE Assignment

Miaowen Dong

August 11, 2026

1 Introduction

In recent years, symmetry learning has drawn growing attention because it applies to many real-world tasks, such as autonomous driving and protein and materials design. An equivariant neural network (NN) encodes symmetry directly into the model, yielding more efficient data usage and better generalization. Two broad approaches exist. An exact equivariant NN is equivariant to the group action by construction, hard-coding the group structure into the architecture. An approximately equivariant NN instead acquires equivariance during training from group-action information, e.g., an augmented dataset; because its architecture is far less constrained, it often generalizes better to unseen group actions and out-of-distribution inputs. My research follows the second route, combining data augmentation with deep ensembles, which are known to give strong equivariance [1]. Deep ensembles, however, are costly at both training and test time. We therefore explore Bayesian techniques that approximate ensemble behavior at lower cost — and, as a by-product, allow us to quantify how equivariant a network actually is [2]. This is where software engineering enters. To compare augmentation, deep ensembles, and their Bayesian approximations, one must decide whether a network is "equivariant enoug" — yet such a network has no per-input ground truth to check against. Quantifying an equivariance defect together with its uncertainty is thus an instance of the classical test oracle problem: building a reliable, interpretable correctness criterion for a system whose expected outputs are unknown. This essay revisits my research through that lens, linking it to metamorphic testing, requirements engineering for AI systems, and two recent papers on testing and verifying ML systems.

2 Lecture principles

Two ideas from lecture slide deck 02 are closely related to my research.

The test oracle problem and metamorphic testing. A test oracle specifies the expected behavior for a given input. Ideally it is generated from a model; more typically it is supplied by a human. The difficulty, in both cases, is defining an oracle that is both expressive and precise — and for ML systems such an oracle is often

unavailable. Metamorphic testing sidesteps this by checking an invariant between the outputs of related inputs rather than any single expected output, e.g. $M(x) = M(y)$. Equivariance, $f(gx) = gf(x)$, is exactly such a metamorphic relation (MR): applying a group action to the input should apply the corresponding action to the output. In my research we define several defect metrics to measure violations of this MR — for example, orbit same prediction (OSP) and a symmetric KL divergence. The OSP is a boolean oracle, returning a same/different verdict; the symmetric KL divergence is not boolean but is far more informative. This distinction matters because my work targets approximate equivariance, where the relation never holds exactly and the violation is inherently nonzero. A boolean MR is therefore the wrong tool: the MR must become graded and probabilistic, which in turn calls for a probabilistic oracle. How to estimate this graded oracle reliably is itself a testing question, which connects to Property-based testing (PBT).

Property-based testing (PBT). The slides also present property-based testing (PBT). One first expresses the expectation/oracle as a parameterised property or "law", and then tests it on randomly generated data to surface unusual cases. Equivariance is naturally such a property rather than a per-example assertion: the law $f(gx) = gf(x)$ is quantified over all inputs x and all group actions g. Our (x, g) pairs are drawn by Monte Carlo sampling across multiple random seeds, which is precisely an estimate of the degree of non-equivariance from randomly generated test data. Applying PBT to a stochastic system, however, requires a rule for deciding when an estimated violation is significant. Our decomposition $\mathcal{E}_T = \mathcal{E}_\infty + V/\sqrt{T}$ addresses exactly this: it separates the true residual non-equivariance $\mathcal{E}_\infty$ from a finite-sample term that vanishes as the number of samples T grows, giving a principled account of how the estimate converges and how precise it is.

3 Guest-lecture principles

Two principles from Frattini's guest lecture relate closely to my research.

Problem/solution space and measurable quality requirements. Requirements engineering (RE) distinguishes the problem space (what the system should do, and why) from the solution space (how it does it). A core RE activity is decomposing an abstract system goal into concrete, measurable quality requirements. Equivariance is a problem-space property: the specification "the network should be equivariant" is independent of how it is achieved (exact hard-coding, approximate learning, or our Bayesian approximation are all solution-space answers). This is why a single equivariance specification can be used to compare different implementations. My defect metrics are an instance of the RE decomposition itself: the abstract goal "be sufficiently equivariant" is operationalised into measurable quantities such as OSP, the symmetric KL divergence, and the $\mathcal{E}_T$ estimate. The interesting difference from

classical RE is not that my target is a network rather than software, but that measurability itself becomes hard: a requirement like "encrypted" is directly checkable, whereas measuring an approximately-satisfied equivariance specification requires sampling and a statistical decision rule — turning the decomposition step from a specification act into an estimation problem (Section 2).

An open challenge: evaluation without ground truth. Frattini also notes that quality priorities shift for AI systems: accuracy and reliability matter more, and retrainability appears as an entirely new quality. He names evaluation without a clear ground truth as a core open challenge. My research is a concrete instantiation of one answer to it: for the specific property of equivariance, a metamorphic relation plus a probabilistic oracle provides a checkable criterion in the absence of a per-input ground truth. This reframes uncertainty quantification: a by-product in my research, it becomes a first-class non-functional requirement from an RE4AI standpoint — without it, one cannot decide whether a violation is significant, and the evaluation-without-ground-truth problem cannot be answered reliably.

4 Data scientists, software engineers, and AI engineers

The book [3] draws a broad distinction: a data scientist is model-centric — searching, in an exploratory and iterative way, for the best-performing model on given data — whereas a software engineer is system-centric, concerned with interfaces, testing, deployment, and keeping a system reliable and maintainable over time. I find the distinction descriptively accurate but not a clean partition of people, and my own work sits across it.

On the relationship between data and model, I see the two as chosen *together* rather than as separate concerns: the right pairing depends on the properties of both. Even within a Bayesian neural network setting, different inference methods demand different amounts of data — variational inference clearly needs more training data than MC dropout, and it requires me to supply more complete equivariance information to reach the same level of learned equivariance. Deciding this pairing is neither purely a modelling choice nor purely an engineering one.

The book's argument that production ML needs a system-wide rather than a model-centric view applies directly to my area. A network with high single-model accuracy can still violate equivariance once it is placed in a full pipeline of augmentation, test-time transforms, and downstream use; equivariance is therefore a *system-level* property, not a model-level score. This is also why deep ensembles are so effective in my research: combining several individually biased models into one system improves behaviour that no single model guarantees — a concrete instance of the system-wide view the book advocates.

I doubt these will remain two distinct roles. During my internship at Infineon I was hired as a data scientist, but as the project progressed I was drawn increasingly into software-engineering work, because a research result inside a company must

be easy for others to use and to maintain after I leave. In that setting the boundary between producing a model and engineering a usable system simply dissolved.

Where does "AI Engineering" fit? I would say the technology is new but the activity is not. That we now cut vegetables with a metal knife instead of a stone one does not make it any less cooking — nor was the caveman not cooking. AI Engineering is, in the main, software engineering for AI systems; what is genuinely new is a narrow core — such as building uncertainty-aware correctness criteria for systems that have no oracle — for which neither traditional software engineering nor traditional data science already has the tools.

Finally, foundation models, AI coding assistants, RAG, prompt pipelines, and agentic workflows mostly *improve the seams* between these roles rather than erasing them. They provide a shared third-party reference when a project is handed over, easing the transition that, in my Infineon experience, was the hardest part of moving a result from research into maintainable production.

5 Paper analysis

5.1 Paper 1: Domain-Validity-Gated Metamorphic Testing of Scientific ML Surrogates

SciML surrogates face the classic oracle problem: for an arbitrary input there is no cheaply available exact output to compare against. [4] uses metamorphic testing to bypass this, but its core contribution is that *a candidate MR is not automatically valid*. A candidate relation must first pass a domain-validity rubric — it needs a physical or software basis, its transformation preconditions must hold, boundary conditions must remain compatible, and it must be numerically decidable, meaning the verdict tolerance dominates the intrinsic error floor $O(h)$ of the measuring operator — before it can be turned into an executable, auditable MR-card. The paper also gives a two-dimensional verdict that separates a model-level violation from a relation applied outside its valid domain. This matters for engineering AI systems because it upgrades the casual practice of "write an invariant and test it" into a quality-assurance process that is reproducible, auditable, and ties every reported number to a tracked artifact — the key to making an ML testing criterion credible.

The paper explicitly treats equivariance as a symmetry MR (Section 3.2), exactly as I treat equivariance as a metamorphic relation. Its central finding in Section 5.3 — that a model with high in-distribution accuracy still severely violates mirror-y equivariance — is published evidence for the claim in my Section 2 that approximate equivariance yields a nonzero violation and therefore requires a graded oracle. This is where my own contribution stands out. Their gate rests on a *static* tolerance that must dominate a fixed operator floor $O(h)$; my criterion instead controls sampling noise, a more tractable target, since sampling noise is directly controllable and in most cases comes with an explicit convergence rate (the $V/\sqrt{T}$ term in $\mathcal{E}_T = \mathcal{E}_\infty + V/\sqrt{T}$). The two are complementary: their floor separates a violation from numerical noise, while mine separates residual non-equivariance from

sampling noise.

I reuse the safety-critical autonomous-driving perception system from the previous paper, where equivariance is a core correctness constraint. Their "MR-card + validity rubric" becomes a test gate in the system's CI: each equivariance specification first passes the rubric (preconditions, tolerance), then executes as an auditable asset. In this pipeline my uncertainty-aware equivariance criterion would replace their static tolerance as the decision core of the gate: rather than a fixed threshold, the admit/reject verdict is driven by whether the estimated violation is statistically significant given the sampling budget.

A natural longer-term extension is to borrow their two-dimensional verdict and make my own two sources of doubt explicit as separate axes — residual non-equivariance on one axis, sampling noise on the other — so that a genuine model-level violation is never confused with an under-sampled estimate.

5.2 Paper 2: An AI-driven Requirements Engineering Framework Tailored for Evaluating AI-Based Software

Traditional RE has been refined for conventional software, but AI-based software (here specifically vision-based perception software) does not fit those practices. [5] proposes the W-AIS framework: it first uses RE methods to express domain concepts (e.g. pedestrian, aircraft) as *partial specifications* — ontology-based (subject, predicate, object) triples — that form a minimum set of requirements the AIS should satisfy; it then uses XAI to measure the distance between the model's perception and these specifications, labels a concept as "under-perceived" when the distance exceeds a threshold, and retrains with targeted supplementary data. This matters because it gives a structured way to evaluate whether an AI satisfies its specifications when per-input ground truth is lacking, directly addressing the RE4AI problem that an AIS cannot be assessed without a reference point.

My work is in the same spirit. The paper uses partial specification + distance threshold to check whether the model meets a specification; I use equivariance MR + defect metric to check whether the network meets an equivariance specification [2]. Both test the internal behaviour of the model against a specification rather than only its end-to-end accuracy. The paper states explicitly that AIS evaluation lacks a reference point and must fall back on human intuition — precisely the "evaluation without ground truth" challenge I picked in Section 3 — and my probabilistic oracle is a more rigorous answer for one class of formalizable property, equivariance. A revealing contrast: W-AIS grounds its specifications in human-perceived semantics and needs a human in the loop to judge under-perception. But an AI can only learn from what humans already know or can label, whereas a mathematical property follows only its own laws. For concepts humans grasp intuitively their semantic approach fits well; but where human perception cannot cleanly separate cases, a formal criterion like equivariance becomes *more* usable, not less — much as mathematical reasoning has historically revealed structure that unaided perception could not (e.g. predicting a planet's existence from orbital mechanics before it

was observed). My work targets exactly this formalizable regime.

In a shared RE4AI pipeline the two guard different *types* of specification: W-AIS checks whether the model perceives the semantic variants of a concept (the semantic layer), while my equivariance criterion checks whether the model is equivariant to geometric transformations (the mathematical layer). My contribution enters at this geometric layer: equivariance can be established in the embedding space, so my criterion checks whether the model's internal representation respects the required group action, complementing W-AIS's semantic check.

Borrowing their closed loop — partial specification, distance threshold, and specification-guided retraining of under-perceived variants — I could extend my criterion from merely *reporting* an equivariance violation to *triggering* targeted augmentation or retraining when the violation exceeds a threshold; since my research already relies on data augmentation, this interface is natural.

6 Evidence, originality, and GenAI use

Main sources. This essay relies on: Robert Feldt's course lecture slides (lecture 02, quality assurance in SE and ML) and Julian Frattini's guest lecture on requirements engineering, both *course/guest lecture material*; Li et al., *Domain-Validity-Gated Metamorphic Testing of Scientific ML Surrogates* (arXiv:2606.17529, 2026), a *preprint*; Barzamini et al., *An AI-driven Requirements Engineering Framework Tailored for Evaluating AI-Based Software* (CAIN 2025), a *peer-reviewed conference paper*; Kästner, *Machine Learning in Production* (CMU), Ch. 1–2, *online textbook chapters*; and Gerken and Kessel, *Emergent Equivariance in Deep Ensembles*, a *peer-reviewed paper*. I trust the peer-reviewed paper and the curated course/textbook material most; I treat the arXiv preprint as credible but unrefereed, relying on its method and its Section 5.3 result, which I checked against the paper itself.

Use of generative AI. I used a generative AI assistant to brainstorm links between the course material and my research, shortlist and fact-check candidate papers (venue, length, year, authorship — this discarded a first candidate that proved to be a short doctoral-symposium paper), build a section outline, translate my notes from Chinese, and copy-edit. All analysis, judgments, and technical claims — equivariance as a metamorphic relation, the graded/probabilistic oracle, the $\mathcal{E}_T = \mathcal{E}_\infty + V/\sqrt{T}$ decomposition, and the positions in Section 4 — are my own; the assistant helped phrase and organise them, not produce them. I take full responsibility for all content, citations, and wording.

References

[1] Jan E. Gerken and Pan Kessel. *Emergent Equivariance in Deep Ensembles*. June 2024. DOI: 10.48550/arXiv.2403.03103. arXiv: 2403.03103 [cs]. (Visited on 09/01/2025).

[2] Miaowen Dong, Axel Flinth, and Jan E. Gerken. *Equivariance and Augmentation for Bayesian Neural Networks*. 2026. arXiv: 2606.26273 [cs.LG]. URL: https://arxiv.org/abs/2606.26273.

[3] *Cover - Machine Learning in Production: From Models to Products*. https://mlip-cmu.github.io/book/. (Visited on 07/18/2026).

[4] Meng Li et al. *Domain-Validity-Gated Metamorphic Testing of Scientific ML Surrogates*. June 2026. DOI: 10.48550/arXiv.2606.17529. arXiv: 2606.17529 [cs.CE]. (Visited on 07/17/2026).

[5] Hamed Barzamini et al. "An AI-driven Requirements Engineering Framework Tailored for Evaluating AI-Based Software". In: *2025 IEEE/ACM 4th International Conference on AI Engineering – Software Engineering for AI (CAIN)*. Apr. 2025, pp. 138–149. DOI: 10.1109/CAIN66642.2025.00025. (Visited on 07/17/2026).